COMPUTER ARCHITECTURE AND ORGANIZATION: PROGRAMMABLE COUNTER PROJECT REPORT

GitHub Repository: https://github.com/alikhedr-cmyk/programmable-counter

Group members :

#	Student Name and Surname	Student ID	Signature
1	Trneem Abdelmaksud	22040102136	
2	Mukhammad Ismailov	23040102093	
3	Esma Selim	22040102159	
4	Muqaddas Samadova	23040102094	
5	Islam Dadaev	23040102092	
6	Shahajina Akter Orpy	24040102038	
7	Nikita Afanasev	23040102128	
8	Gehad Ali Khedr	23040102086	

1. Project Overview

This project implements a programmable counter using Verilog HDL. The counter supports hold, up-counting, down-counting, and parallel load operations. The design follows a modular control and datapath architecture, which makes the hardware component easier to understand, test, and integrate.

2. Digital Design

The digital design is divided into a control path and a datapath. The control path decodes the selected mode and generates operation requests. The datapath updates the counter value and produces the final output flags.

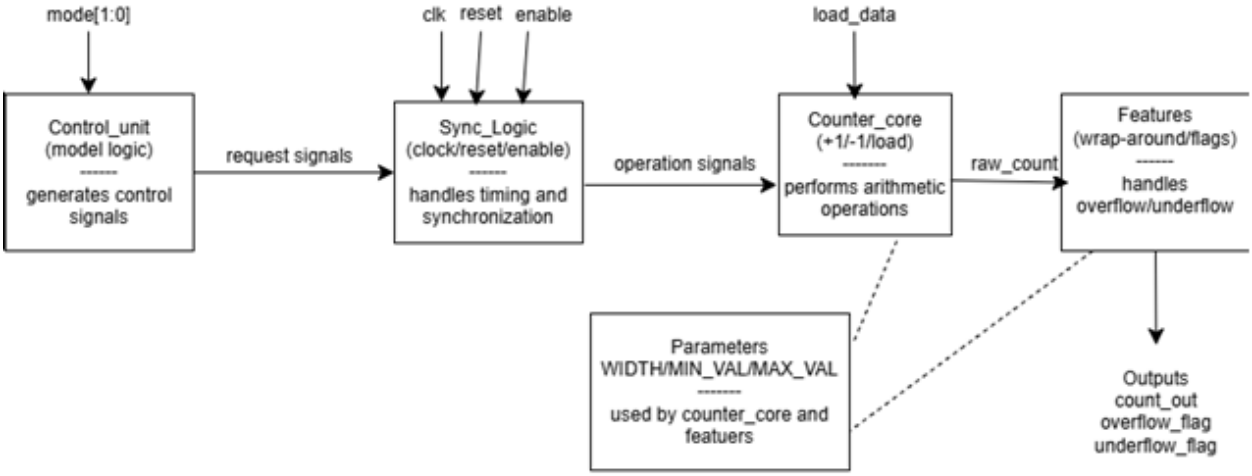


Figure 1. Programmable counter block diagram.

The Control Unit receives the 2-bit mode input and generates request signals for hold, up, down, or load operations. The Sync Logic combines these request signals with reset and enable, then sends operation signals to the Counter Core. The Counter Core performs the arithmetic operation and produces raw_count. The Features module passes the final count and generates overflow and underflow flags. The design uses WIDTH, MAX_VAL, and MIN_VAL parameters to make the counter configurable.

Mode encoding

Mode	Operation	Description
2'b00	Hold	Keep the current counter value
2'b01	Up	Increment the counter
2'b10	Down	Decrement the counter
2'b11	Load	Load the parallel input value

Input & output

Signal	Direction	Description
clk	input	Clock signal used to update the counter
reset	input	Resets the counter value
enable	input	Enables or disables counter operation
mode[1:0]	input	Selects hold, up, down, or load operation
load_data	input	Parallel value loaded into the counter
count_out	output	Final counter value
overflow_flag	output	Indicates overflow condition
underflow_flag	output	Indicates underflow condition

3. Verilog Implementation

Control Unit (control_unit.v)

The control_unit acts as the instruction decoder of the programmable counter. Its role is to read the 2-bit mode input and generate one request signal at a time, such as req_up, req_down, req_load, or req_hold. The module is implemented using pure combinational logic, which allows immediate decoding without depending on the clock.In the updated version, the control unit uses the shared params.vh file instead of hardcoded mode values. This improves consistency across the project and makes the design easier to maintain. The outputs are initialized at the start of the always block to prevent latch generation, and the default case forces the system into hold mode for safety.

Arithmetic Core (counter_core.v)

The counter_core is the execution engine of the system. It performs the actual counting and loading operations according to the control signals provided by the synchronization logic. The module is implemented using behavioral Verilog and supports a parameterized WIDTH, allowing the counter to be scaled to different bit sizes. The operation priority is organized so that reset has the highest priority, followed by load, then increment/decrement, while hold preserves the current value. This structure ensures predictable behavior and avoids control conflicts during execution.

Advanced Features (features.v)

The features module enhances the basic counter by handling terminal-count behavior. It implements wrap-around logic so that the counter returns to the minimum value after reaching the maximum, and vice versa during down-counting. In addition, it generates overflow_flag and underflow_flag signals to indicate when the counter crosses its valid boundaries. These features make the design more robust and flexible, especially for programmable and reusable digital systems. The module is also parameterized using WIDTH, MAX_VAL, and MIN_VAL, which allows it to work with different counter sizes.

sync_logic.v

The sync_logic module acts as the interface between the control unit and the counter core. Its main role is to control when the requested operation is allowed to execute by using the reset and enable signals. When reset is active or enable is low, the counter remains in the hold state. Otherwise, the module passes the requested operation safely to the counter core, with load having higher priority than up and down. This separation improves the organization of the design and keeps the timing and control conditions independent from the arithmetic logic.

params.vh

The project uses shared constants and configurable width settings to improve flexibility and code organization. The params.vh file defines common values such as the operation modes and default limits, which reduces repeated hardcoded values across the design. In addition, width-dependent definitions make the counter easier to scale and maintain. This approach improves readability, supports reuse, and makes the design more suitable for future extensions.

top_module.v

The top_module is responsible for integrating all project modules into one complete system. It connects the control_unit, sync_logic, counter_core, and features modules using internal wires and shared signals. This module represents the final structure of the programmable counter and ensures that all parts work together correctly.

In addition to module integration, this part also includes project organization on GitHub and preparation for final testing and reporting. The top-level design passes the mode, reset, enable, and load inputs through the proper modules, then produces the final counter output along with overflow and underflow flags. This integration step is important because it transforms the separate modules into one functional hardware design.

4. Testbench and Simulation

Unit Testing and Verification

A bottom-up verification approach was used to validate the control_unit and counter_core independently before system integration. Directed stimulus was applied through dedicated unit testbenches to check both normal and boundary cases. For the control_unit, a full truth-table sweep was performed to confirm correct mode decoding. For the counter_core, simulations verified reset behavior, load operation, increment, decrement, and boundary transitions such as overflow and underflow in a 4-bit configuration. The observed simulation results matched the expected outputs, indicating that the tested modules were stable and ready for integration.

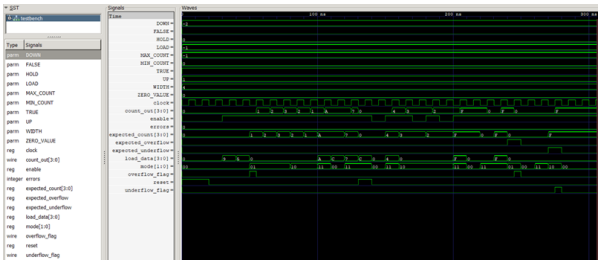
testbench.v

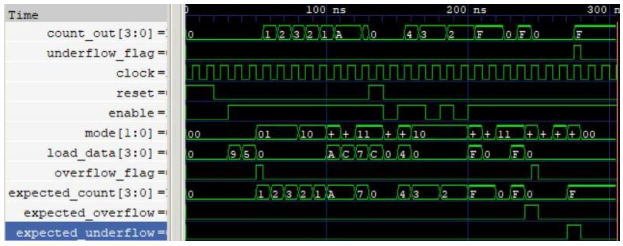
The testbench is designed to verify the functional correctness of a parameterized counter module (top_module) across multiple operating modes including hold, up-count, down-count, and load. It generates a 10ns clock signal and applies stimulus using a structured task (apply_and_check) that drives inputs on the negative edge and checks outputs on the positive edge. A parallel reference model is implemented inside the testbench to compute expected counter values as well as overflow and underflow behavior. The design includes systematic test sequences for reset behavior, boundary conditions, wrap-around functionality, and enable toggling scenarios. Any mismatch between expected and actual outputs is logged as an error, while results are also captured in a VCD waveform file for debugging.

5. GTKWave Results

All tests were passed successfully, the program works as intended.
The test log:

```
[16000] PASS count = 0 overflow = 0 underflow = 0
[26000] PASS count = 0 overflow = 0 underflow = 0
---- Test 1: Hold ----
[36000] PASS count = 0 overflow = 0 underflow = 0
[46000] PASS count = 0 overflow = 0 underflow = 0
---- Test 2: Up ----
[56000] PASS count = 1 overflow = 0 underflow = 0
[66000] PASS count = 2 overflow = 0 underflow = 0
[76000] PASS count = 3 overflow = 0 underflow = 0
---- Test 3: Down ----
[86000] PASS count = 2 overflow = 0 underflow = 0
[96000] PASS count = 1 overflow = 0 underflow = 0
---- Test 4: Load ----
[106000] PASS count = 10 overflow = 0 underflow = 0
[116000] PASS count = 10 overflow = 0 underflow = 0
---- Test 5: Reset during load. ----
[126000] PASS count = 7 overflow = 0 underflow = 0
[136000] PASS count = 0 overflow = 0 underflow = 0
[146000] PASS count = 0 overflow = 0 underflow = 0
---- Test 6: Enable toggling during dowm-count. ----
[156000] PASS count = 4 overflow = 0 underflow = 0
[166000] PASS count = 3 overflow = 0 underflow = 0
[176000] PASS count = 3 overflow = 0 underflow = 0
[186000] PASS count = 2 overflow = 0 underflow = 0
[196000] PASS count = 2 overflow = 0 underflow = 0
---- Test 7: Max/min terminal behavior. ----
[206000] PASS count = 15 overflow = 0 underflow = 0
[216000] PASS count = 15 overflow = 0 underflow = 0
[226000] PASS count = 0 overflow = 0 underflow = 0
---- Test 8: Wrap-around and flags. ----
[236000] PASS count = 15 overflow = 0 underflow = 0
[246000] PASS count = 0 overflow = 1 underflow = 0
[256000] PASS count = 0 overflow = 0 underflow = 0
[266000] PASS count = 0 overflow = 0 underflow = 0
[276000] PASS count = 15 overflow = 0 underflow = 1
[286000] PASS count = 15 overflow = 0 underflow = 0
===================================
Testbench passed.
```





6. Verilog Code

control_unit.v

```
module control_unit (
        input  wire [1:0] mode,

        output reg req_up,
        output reg req_down,
        output reg req_load,
        output reg req_hold
);
        always @(*) begin
        req_up   = 1'b0;
        req_down = 1'b0;
        req_load = 1'b0;
        req_hold = 1'b0;

        case (mode)
        2'b00: req_hold = 1'b1;
        2'b01: req_up   = 1'b1;
        2'b10: req_down = 1'b1;
        2'b11: req_load = 1'b1;
        default: req_hold = 1'b1;
        endcase
        end
endmodule
```

sync_logic.v

```
module sync_logic(
        input  wire clk,
        input  wire reset,
        input  wire enable,
        input  wire req_up,
        input  wire req_down,
        input  wire req_load,
        input  wire req_hold,

        output wire core_reset,
        output reg  do_up,
        output reg  do_down,
        output reg  do_load,
        output reg  do_hold
);
        assign core_reset = reset;

        always @(*) begin
        do_up   = 1'b0;
        do_down = 1'b0;
        do_load = 1'b0;
        do_hold = 1'b1;

        if (reset) begin
        do_hold = 1'b1;
        end else if (!enable) begin
        do_hold = 1'b1;
        end else if (req_load) begin
        do_hold = 1'b0;
        do_load = 1'b1;
        end else if (req_up) begin
        do_hold = 1'b0;
        do_up   = 1'b1;
        end else if (req_down) begin
        do_hold = 1'b0;
        do_down = 1'b1;
        end else if (req_hold) begin
        do_hold = 1'b1;
        end
        end
endmodule
```

counter_core.v

```
module counter_core #(
        parameter WIDTH = 4
)(
        input  wire                 clk,
        input  wire                 core_reset,
        input  wire                 do_up,
        input  wire                 do_down,
        input  wire                 do_load,
        input  wire [WIDTH-1:0]     load_data,
        output reg  [WIDTH-1:0]     count
);
        always @(posedge clk or posedge core_reset) begin
        if (core_reset) begin
        count <= {WIDTH{1'b0}};
        end else if (do_load) begin
        count <= load_data;
        end else if (do_up) begin
        count <= count + 1'b1;
        end else if (do_down) begin
        count <= count - 1'b1;
        end else begin
        count <= count;
        end
        end

endmodule
```

features.v

```
module features #(
parameter WIDTH   = 4,
        parameter MAX_VAL = (1 << WIDTH) - 1,
        parameter MIN_VAL = 0
)(
        input  wire [WIDTH-1:0] raw_count,
        input  wire             do_up,
        input  wire             do_down,
        output reg  [WIDTH-1:0] final_count,
        output reg              overflow_flag,
        output reg              underflow_flag
);
        always @(*) begin

        final_count = raw_count;
        overflow_flag = 1'b0;
        underflow_flag = 1'b0;

        if (do_up && raw_count == MIN_VAL[WIDTH-1:0]) begin
        overflow_flag = 1'b1;
        end

        if (do_down && raw_count == MAX_VAL[WIDTH-1:0]) begin
        underflow_flag = 1'b1;
        end
        end

endmodule
```

params.vh

```
`ifndef PARAMS_VH

`define PARAMS_VH

`define DEFAULT_WIDTH 4

`define MODE_HOLD 2'b00

`define MODE_UP   2'b01

`define MODE_DOWN 2'b10

`define MODE_LOAD 2'b11

`define DEFAULT_MIN {`DEFAULT_WIDTH{1'b0}}

`define DEFAULT_MAX {`DEFAULT_WIDTH{1'b1}}

`endif
```

counter_core_tb.v

```
`timescale 1ns/1ps


module tb_counter_core();


        parameter WIDTH = 4;

        reg clk, core_reset, do_up, do_down, do_load;
```

```
	reg [WIDTH-1:0] load_data;
	wire [WIDTH-1:0] count;

	counter_core #(.WIDTH(WIDTH)) uut (
	.clk(clk), .core_reset(core_reset),
	.do_up(do_up), .do_down(do_down), .do_load(do_load),
	.load_data(load_data), .count(count)
	);
	always #5 clk = ~clk;

	initial begin

	clk = 0; core_reset = 0; do_up = 0; do_down = 0; do_load = 0; load_data =
0;

	$display("Starting Counter Core Unit Test...");

	core_reset = 1; #15;
	core_reset = 0;
	if (count === 4'b0000) $display("[PASS] Reset Successful");

	load_data = 4'hA; do_load = 1; @(posedge clk);
	#1; do_load = 0;
	if (count === 4'hA) $display("[PASS] Parallel Load Successful");

	do_up = 1; repeat(2) @(posedge clk);
	#1; do_up = 0;
	if (count === 4'hC) $display("[PASS] Increment (+2) Successful");

	load_data = 4'h0; do_load = 1; @(posedge clk);
	#1; do_load = 0; do_down = 1; @(posedge clk);
	#1;
	if (count === 4'hF) $display("[PASS] Underflow (0->F) Successful");

	$display("Counter Core Testing Complete.");
	$finish;
	end
endmodule
```

control_unit_tb.v

```
`timescale 1ns/1ps

module tb_control_unit(); reg  [1:0] mode; wire req_up, req_down, req_load, req_hold;
 control_unit uut (
	.mode(mode),
	.req_up(req_up),
	.req_down(req_down),
	.req_load(req_load),
	.req_hold(req_hold)
 );
 initial begin
	$display("Starting Control Unit Unit Test...");

	mode = 2'b00; #10;
	if (req_hold === 1) $display("[PASS] Mode 00: Hold Active");

	mode = 2'b01; #10;
	if (req_up === 1)   $display("[PASS] Mode 01: Up Active");

	mode = 2'b10; #10;
	if (req_down === 1) $display("[PASS] Mode 10: Down Active");

	mode = 2'b11; #10;
	if (req_load === 1) $display("[PASS] Mode 11: Load Active");

	$display("Control Unit Testing Complete.");
	$finish;
 end

endmodule
```

top_module.v

```
module top_module #(

	parameter WIDTH   = 4,
	parameter MAX_VAL = (1 << WIDTH) - 1,
	parameter MIN_VAL = 0
)(
	input  wire                 clk,
	input  wire                 reset,
	input  wire                 enable,
	input  wire [1:0]           mode,
	input  wire [WIDTH-1:0]     load_data,
	output wire [WIDTH-1:0]     count_out,
	output wire                 overflow_flag,
	output wire                 underflow_flag
);

	wire req_up;
	wire req_down;
	wire req_load;
	wire req_hold;

	wire core_reset;
	wire do_up;
	wire do_down;
	wire do_load;
	wire do_hold;

	wire [WIDTH-1:0] raw_count;
	wire [WIDTH-1:0] final_count;

	control_unit u_control_unit (
	.mode(mode),
	.req_up(req_up),
	.req_down(req_down),
	.req_load(req_load),
	.req_hold(req_hold)
	);

	sync_logic u_sync_logic (
	.clk(clk),
	.reset(reset),
	.enable(enable),
	.req_up(req_up),
	.req_down(req_down),
	.req_load(req_load),
	.req_hold(req_hold),
.core_reset(core_reset),
	.do_up(do_up),
	.do_down(do_down),
	.do_load(do_load),
	.do_hold(do_hold)
	);

	counter_core #(
	.WIDTH(WIDTH)
	) u_counter_core (
	.clk(clk),
.core_reset(core_reset),
	.do_up(do_up),
	.do_down(do_down),
	.do_load(do_load),
	.load_data(load_data),
	.count(raw_count)
	);

	features #(
	.WIDTH(WIDTH),
	.MAX_VAL(MAX_VAL),
	.MIN_VAL(MIN_VAL)
	) u_features (
	.raw_count(raw_count),
	.do_up(do_up),
	.do_down(do_down),
.final_count(final_count),
.overflow_flag(overflow_flag),
.underflow_flag(underflow_flag)
	);

	assign count_out = final_count;

endmodule
```

testbench.v

```
 `timescale 1ns/1ps


module testbench;

	parameter WIDTH = 4;

	reg clock;
	reg reset;
	reg enable;
	reg [1:0] mode;
	reg [WIDTH-1:0] load_data;
	wire [WIDTH-1:0] count_out;
	wire overflow_flag;
	wire underflow_flag;

	localparam FALSE = 1'b0;
	localparam TRUE  = 1'b1;

	localparam [1:0]
	HOLD = 2'b00,
	UP   = 2'b01,
	DOWN = 2'b10,
	LOAD = 2'b11;

	localparam [WIDTH-1:0]
	ZERO_VALUE  = {WIDTH{1'b0}},
	MIN_COUNT = {WIDTH{1'b0}},
	MAX_COUNT = (1 << WIDTH) - 1;

	reg [WIDTH-1:0] expected_count;
	reg expected_overflow;
	reg expected_underflow;

	integer errors;

	top_module #(
	.WIDTH(WIDTH),
	.MAX_VAL((1 << WIDTH) - 1),
	.MIN_VAL(0)
	) unit_under_test (
	.clk(clock),
	.reset(reset),
	.enable(enable),
	.mode(mode),
	.load_data(load_data),
	.count_out(count_out),
.overflow_flag(overflow_flag),
.underflow_flag(underflow_flag)
```

```
);

initial begin
clock = 1'b0;
forever #5 clock = ~clock;
end

task apply_and_check;
input reset_local;
input enable_local;
input [1:0] mode_local;
input [WIDTH-1:0] load_data_local;
reg [WIDTH-1:0] next_count_local;
begin
@(negedge clock);
reset = reset_local;
enable = enable_local;
mode = mode_local;
load_data = load_data_local;

next_count_local = expected_count;
expected_overflow = FALSE;
expected_underflow = FALSE;

if (reset_local) begin
next_count_local = MIN_COUNT;
end
else if (enable_local) begin
case (mode_local)
HOLD: begin
next_count_local = expected_count; // Hold.
end

UP: begin
if (expected_count == MAX_COUNT) begin
next_count_local = MIN_COUNT;
expected_overflow = TRUE;
end
else begin
next_count_local = expected_count + 1'b1;
end
end

DOWN: begin
if (expected_count == MIN_COUNT) begin
next_count_local = MAX_COUNT;
expected_underflow = TRUE;
end
else begin
next_count_local = expected_count - 1'b1;
end
end

LOAD: begin
next_count_local = load_data_local;
end

default: begin
next_count_local = expected_count;
end
endcase
end

expected_count = next_count_local;

@(posedge clock);
#1;

if ((count_out !== expected_count) || (overflow_flag !==
expected_overflow) || (underflow_flag !== expected_underflow)) begin
errors = errors + 1;
$display("[%0t] FAIL exp: count = %0d overflow = %b underflow = %b | got:
count = %0d overflow = %b underflow = %b", $time, expected_count,
expected_overflow, expected_underflow, count_out, overflow_flag, underflow_flag);
end
else begin
$display("[%0t] PASS count = %0d overflow = %b underflow = %b", $time,
count_out, overflow_flag, underflow_flag);
end
end
endtask

initial begin
$dumpfile("counter_wave.vcd");
$dumpvars(0, testbench);

errors = 0;

reset = TRUE;
enable = FALSE;
mode = HOLD;
load_data = ZERO_VALUE;

expected_count = MIN_COUNT;
expected_overflow = FALSE;
expected_underflow = FALSE;

/* Reset */
apply_and_check(TRUE,  FALSE, HOLD, ZERO_VALUE);
apply_and_check(FALSE, FALSE, HOLD, ZERO_VALUE);

/* 1. Hold */
$display("---- Test 1: Hold ----");
apply_and_check(FALSE, TRUE, HOLD, 4'd9);
apply_and_check(FALSE, TRUE, HOLD, 4'd5);

/* 2. Up */
$display("---- Test 2: Up ----");
apply_and_check(FALSE, TRUE, UP, ZERO_VALUE);
apply_and_check(FALSE, TRUE, UP, ZERO_VALUE);
apply_and_check(FALSE, TRUE, UP, ZERO_VALUE);

/* 3. Down */
$display("---- Test 3: Down ----");
apply_and_check(FALSE, TRUE, DOWN, ZERO_VALUE);
apply_and_check(FALSE, TRUE, DOWN, ZERO_VALUE);

/* 4. Load */
$display("---- Test 4: Load ----");
apply_and_check(FALSE, TRUE, LOAD, 4'd10);
apply_and_check(FALSE, TRUE, HOLD, 4'd12);

/* 5. Reset during load. */
$display("---- Test 5: Reset during load. ----");
apply_and_check(FALSE, TRUE, LOAD, 4'd7);
apply_and_check(TRUE,  TRUE, LOAD, 4'd12);
apply_and_check(FALSE, FALSE, HOLD, ZERO_VALUE);

/* 6. Enable toggling during down-count. */
$display("---- Test 6: Enable toggling during dowm-count. ----");
apply_and_check(FALSE, TRUE, LOAD, 4'd4);
apply_and_check(FALSE, TRUE, DOWN, ZERO_VALUE);
apply_and_check(FALSE, FALSE, DOWN, ZERO_VALUE);
apply_and_check(FALSE, TRUE, DOWN, ZERO_VALUE);
apply_and_check(FALSE, FALSE, DOWN, ZERO_VALUE);

/* 7. Max/min terminal behavior. */
$display("---- Test 7: Max/min terminal behavior. ----");
apply_and_check(FALSE, TRUE, LOAD, MAX_COUNT);
apply_and_check(FALSE, TRUE, HOLD, ZERO_VALUE);
apply_and_check(FALSE, TRUE, LOAD, MIN_COUNT);

/* 8. Wrap-around and flags. */
$display("---- Test 8: Wrap-around and flags. ----");
apply_and_check(FALSE, TRUE, LOAD, MAX_COUNT);
apply_and_check(FALSE, TRUE, UP, ZERO_VALUE); // Underflow
expected.
apply_and_check(FALSE, TRUE, HOLD, ZERO_VALUE); // Flag must
clear on hold.

apply_and_check(FALSE, TRUE, LOAD, MIN_COUNT);
apply_and_check(FALSE, TRUE, DOWN, ZERO_VALUE); //
Underflow expected.
apply_and_check(FALSE, TRUE, HOLD, ZERO_VALUE); // Flag must
clear on hold.

$display("====================================");
if (errors == 0) begin
$display("Testbench passed.");
end
else begin
$display("Testbench failed: %0d error(s)", errors);
end
$display("====================================");

#20;
$finish;
end

endmodule
```

7. Conclusion

The programmable counter demonstrates a modular digital hardware design using Verilog HDL. The design supports multiple counting operations and is verified using testbenches and waveform analysis.